BoxPuzzle.java

Pattern: Composite

Justification: Item and Crate implement the same interface, allowing uniform treatment. Crate contains Units and delegates size() calls recursively, enabling tree structures.

DailyChoice.java

Pattern: Strategy

Justification: Multiple FeeRule implementations define interchangeable algorithms. CheckoutDesk delegates to a rule and can switch it at runtime.

GateMood.java

Pattern: State

Justification: Door holds a Mode and delegates behavior. Each Mode changes the Door's state, altering behavior dynamically.

KitchenPlan.java

Pattern: Template Method

Justification: Meal defines a fixed algorithm in prepare(), while subclasses override cook(), customizing steps via inheritance.

MorningCoffee.java

Pattern: Decorator

Justification: ExtraCup wraps another Cup and delegates calls, adding behavior dynamically through composition.

NightShift.java

Pattern: Factory Method

Justification: Supervisor defines workflow but defers object creation to subclasses via hire().

OldSpeakerPort.java

Pattern: Adapter

Justification: Wire adapts LegacyPlayer to SoundSource by translating method calls.

OnlyOneMic.java

Pattern: Singleton

Justification: Mixer has a private constructor and a single static instance returned globally.

PrivateShelf.java

Pattern: Proxy

Justification: CheckedArchive controls access to RealArchive and delegates conditionally.

RemoteClick.java

Pattern: Command

Justification: Step objects encapsulate actions and are executed by Button, decoupling invoker and receiver.

TripStarter.java

Pattern: Facade

Justification: TripButton provides a simple interface to multiple subsystems.

WatchTower.java

Pattern: Observer

Justification: WeatherCenter notifies multiple Screen observers when state changes.